

Measuring the time efficiency of development and the execution speed of the code between C and Python through controlled tasks

David Gaby Sahala Sinambela
Artificial Intelligence
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
david.sinambela@binus.ac.id

Fiko Alexie van Houten
Artificial Intelligence
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
fiko.houten@binus.ac.id

Richtjhie Hartawan Agusta
Artificial Intelligence
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
richtjhie.agusta@binus.ac.id

Said Achmad
Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
said.achmad@binus.edu

Maulin Nasari
Computer Science Department,
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
maulin.nasari@binus.edu

I. INTRODUCTION

The selection of a programming language inherently involves a trade-off between development efficiency and execution speed. In practice, this often means a trade-off: low-level languages like C deliver superior execution speed and fine-grained hardware control, whereas high-level languages like Python enable much faster coding and prototyping [1] [2]. For example, one study notes that C code “runs under a compiler” and thus “code execution is faster” than in Python, whose interpreter and garbage collector impose additional runtime overhead [1][2]. Conversely, Python programs tend to be far shorter and easier to write as Rohit Kumar and Chander observe, “Python programs are easier to learn, write, and read,” and the development process in Python is generally much faster than in C [1] [3][5]. So, the question arises, does the hidden cost of C’s manual memory management and harder syntaxes make C less superior to Python’s convenience that sacrifices extra CPU power and energy cost? [1] [2][4][18].

The cause of these differences is the language’s execution models. The source code of C is translated directly into machine code before running, which allows the execution to run directly on the CPU with minimal overhead [2][4][14][20]. In contrast, Python’s source code is translated into bytecodes, which then a Virtual Machine executes at runtime, making it slower cause it incurs overhead from the interpretation layer and runtime checks. These abstractions make Python very, very, developer-friendly [5][23] but there’s a significant increase in execution latency [1] [2][19].

We conduct this study, primarily, to find the correlation between development effort and computational output to provide a framework for both C and Python. The main objective is to evaluate whether the reduced development time offered by Python justifies the sacrifice from execution speed [1] [2][6][9]. To achieve this, the research aims to make a methodology using controlled tasks, where there are identical challenges of algorithm programming from simple data processing to complex resource-intensive computations for both C and Python [2][6][9][12]. The goal of this paper is to figure out a data framework so that engineers could then

decide whether to use Python’s fast development with the slower execution speed, or C’s insanely fast runtime justifies the slower development efforts.

This research primary contribution is in the design of a controlled, reproducible experiment that can simultaneously evaluate the development time to research and write the code, but also the runtime efficiency, within the same tasks and the same conditions [6][12][17]. By using these two dimensions, which could be seen in prior works has treated as separate concerns, this research hopes to establish a coherent empirical reference point for programming language decisions. Furthermore, because the methodology is documentable but also reproducible, the framework could be used for other researchers that may apply different language pairs or different hardware environments [6][12].

II. LITERATURE REVIEW

The relationship between programming language choice and developer productivity has attracted steady research attention for decades, and the broad strokes of what the literature says are consistent. Compiled languages run faster. High-level languages are written faster. The gap between them is real, and context dependent. What’s much harder to find is research that puts both dimensions on the same scale at the same time.

1. Starting with the performance side: C and C++ have consistently dominated runtime benchmarks across virtually every domain studied. Kumar’s comparison nature – there’s no virtual machine, no interpreter layer, and no garbage collector consuming CPU cycles during execution [7]. Patel and Kishorbbhai both argue that C’s relevance in modern computing isn’t just historical sentiment; it’s a direct consequence of how the language maps to hardware, giving programmers explicit control over memory and processor usage that interpreted languages simply can’t match [14][20]. For systems where predictability matters as much as speed – operating systems, embedded firmware, real-time applications – that level of control is often non-negotiable.

Empirical benchmark studies reinforce this consistently. Rysak ran a direct comparison of C and Python across several classical algorithm implementations and found C’s execution times superior across the board, with the margin growing as problem complexity increased [19]. Zhang’s 2024 study on topological sorting algorithms is particularly relevant to the experiment – it found that C++ (which shares C’s compiled execution mode) significantly outperformed Python across nearly all test cases, and explicitly called for more research into what it actually costs developers to write the faster implementation [12]. Shoaib extended this to a broader class of classical algorithms and found the same pattern; compiled languages win on execution, regardless of which algorithm is being tested [28].

The energy dimension reinforces the execution argument. Pereira and the others state that large-scale ranking of 27 programming languages by energy efficiency placed C at the top of every category – execution speed, energy consumption, and memory usage – while Python ranked near the bottom in execution efficiency [18]. Stoic and their research team pushed this investigation further in 2025 by measuring not just CPython but eight alternative Python compilation strategies, finding that even Numba and PyPy, the most optimized option. Still consume more energy than C code [4].

Crucially, nearly all prior work treats development effort and runtime efficiency separately the literature is rich in execution benchmarks and energy studies, and it contains numerous qualitative accounts of developer productivity, but it rarely quantifies the joint trade-off [6][17]. That is because few studies have empirically measured how many developers’ hours or how much cognitive effort are saved by writing in Python versus how many machine-milliseconds are lost because of slower execution [6][17]. Inspired by prior algorithmic comparisons [6][13].

Most of the studies or the references that the author uses for this research paper are comparable between two languages. Tao and Zhang et al.’s study of programming assignment time efficiency and using Controlled Task methodology is the closest prior work that we’re doing here. It measured how long and impactful of the human efforts to complete algorithmic assignments in different languages and found that high-level languages users finish faster and compute the performance benchmark of the used programming language but produce less efficient solutions [9][17]. That’s almost exactly the trade-off we’re trying to quantify

III. METHODOLOGY

A. Overview of the Method

This is a quantitative experiment using Controlled Task methods. The core principle is to measure the trade-off between writing code and running code at the same time on the same problems and under the same conditions [6][9][17]. The research method can be summarized as follows:

Method	Description
Benchmarking	Execution time (ms) and memory usage (MB) are measured under standardized and repeatable

	conditions using identical inputs
Comparative Study	Both languages (C and Python) implement the same algorithmic tasks to enable direct comparison
Empirical Measurement	Development time (minutes) and Lines of Code (LoC) are recorded objectively during implementation
Controlled Experiment	All external variables (hardware, OS, input data, and task type) are held constant to isolate programming language as the independent variable

B. Research Design and Variables

The independent variable in this study is the programming language: C and Python. The dependent variables are development (measured in minutes), Lines of Code, and execution speed of both solutions (C and Python). All other factors such as the hardware used for benchmarking, the problem specifications, and test inputs are held constant as control variables in the experiment [7][9][12]

C. Volunteer / Subject

Volunteer programmers as experimental subjects, so the development time data reflects actual programmer behavior rather than one researcher’s performance. Experimental subjects volunteer programmers must have prior experience with both C and Python programming. Volunteers are informed of the study’s purpose and provide their consent to participate. All data collected from volunteers is anonymized and stored securely; no personally identifiable information is reported in the study’s results [17].

D. Problem Structure

To ensure the comparison is fair and generalizable across different types of computational challenges, the experiment is structured around nine problems organized into three difficulty levels:

- Level 1 (Easy): Basic syntax, simple logic
- Level 2 (Medium); Intermediate algorithms, basic data structures
- Level 3 (Hard): Complex logic, algorithms, or computationally intensive calculations

Each volunteer solves all nine problems once in C and once in Python. Volunteers are allowed to use tools such as w3schools and GeekForGeeks to help them develop the code; volunteer is not allowed to use AI [17].

E. Data Collection

- **Input Data Generation**

Large-scale data will be made using AI. For example, both the C and Python algorithms will be using generated data as the input to ensure fairness in the comparison. [1][9][12]

- **Development Metrics (Primary Data)**

- **Development Time:** Measured in minutes using a stopwatch, from blank file to successful execution and passing test cases.
- **Lines of Code (LoC):** Counted after removing blank lines and comments.

- **Execution Metrics (Primary Data)**

- **Execution Time:** Measured in milliseconds using language-native timing functions
- Each program is executed 5 times, and the average is used to reduce variance [14][17]

- **Hardware Environment**

All tests are conducted on a single machine with either a virtual machine or one of the researcher's laptops (as of now, it is still undecided which hardware to use for effective data collecting). Regardless the final decision will remain constant throughout the experiment to maintain fairness and consistency [7][11]

F. Experimental Pipeline

This experiment uses a structured six step pipeline. Firstly, the volunteer receives a problem statement. Secondly, the timer starts and they begin coding from a blank file. Thirdly, the timer stops when the solution for each problem (one solution for each problem per language) is finished. Fourthly, development time is recorded and LoC is counted. Fifthly, the submitted code is collected and sent to the author. Sixth, after all sessions are complete, the team runs each submitted program on the benchmark machine and records execution time and memory usage.

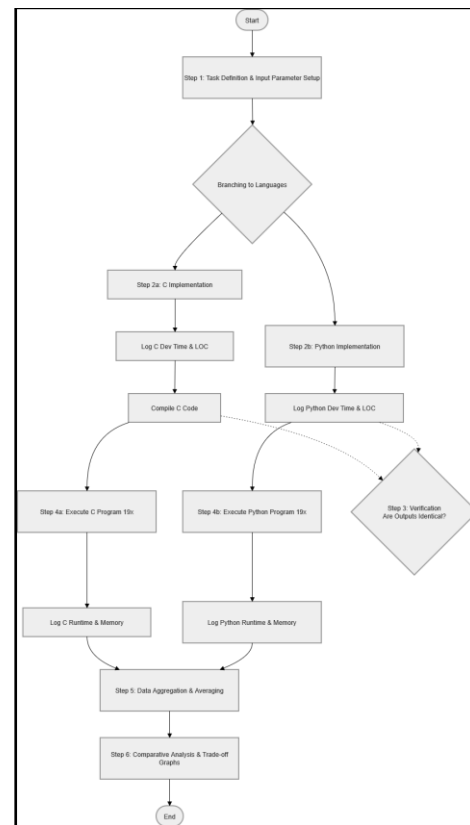


Figure 1: Experimental Workflow Pipeline

G. Validity and Reproducibility

The internal validity is attained by using identical problem specifications and test case sets for both C and Python versions of every problem. A solution that doesn't pass the test cases is returned to the volunteer for correction before it's accepted. Whereas external validity is supported by the three-level difficulty of design. For reproducibility, all of the problem statements, test cases, anonymized source submissions, and raw timing logs will be deposited in a public repository (ex. GitHub) along with the benchmark machine's full hardware and software specifications

IV. RESULT AND DISCUSSION

4.1 Data Overview

This chapter presents our empirical examinations from our controlled comparison between C and Python from 9 different algorithmic problems with varying difficulty. First, we must look at the scope of the collected data.

4.1.1 Dataset Composition:

- **Participants:** 8 volunteers (T01-T08), undergraduate computer scientists from Binus that have practical experience with both C and Python
- **Problems:** 9 algorithmic tasks
 - Easy (E1, E2, E3): basic tasks and logic.
 - Medium (M1, M2, M3): Intermediate algorithms and data structure.
 - Hard (H1, H2, H3): Complex logic and computationally hard tasks to run.
- **Languages:** C and Python
- **Primary Variables:**
 - **Development time (seconds):** a stopwatch from blank file to finished solution.

- Lines of Code (LoC): Executable lines, excluding comments.
- Execution speed (ms): Runtime performance measured over 5 repeated runs per config

4.1.2 Data Completeness

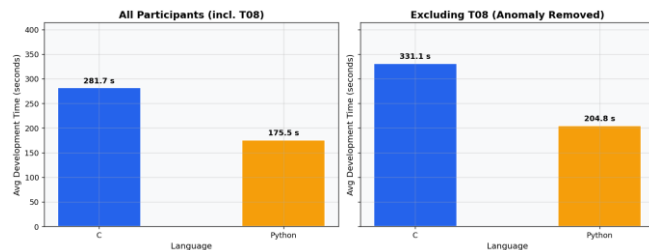
A total of 108 data was collected. We also decided for execution speed benchmarks to use two representative problems: H1 (which is sorting) and H2 (which is a matrix transpose), each tested with three input sizes (100, 5000, and 10000 data points for H1; 100x100, 500x500, and 1000x1000 matrices for H2).

4.2 Development Time Analysis

4.2.1 Overall Development Time

One of the very first things we found was that across 108 theres a significant gap in the development time, Python required less time to code than C.

4.2.1 — Average Development Time: C vs Python



Key Findings:

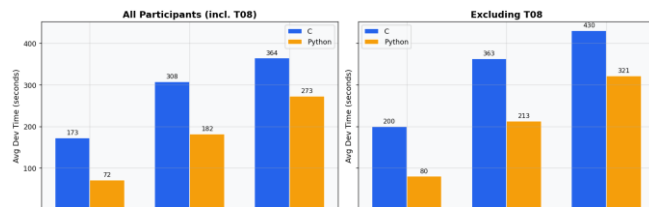
- Mean development time (including T08):
 - **C** (~5.3 minutes)
 - **Python** (~3.4 minutes)
- Mean development time (excluding T08):
 - **C** (~6.1 minutes)
 - **Python** (~3.8 minutes)

(Python is 1.6x times faster than C)

This result aligns very well with prior researches from Kumar et al. [3] and Tao et al. [17], which states due to Python's higher-level syntax and build-in data structure, makes it as though it reduces the cognitive loads needed to implement algorithms. Whereas C requires explicit memory management, type declaration, and manual array handling tends to give friction to the development. Now you may also notice that the graph is divided into 2 parts, with T08 and without T08, which will be discussed shortly after.

4.2.2 Impact of Difficulty

4.2.2 — Average Development Time by Difficulty



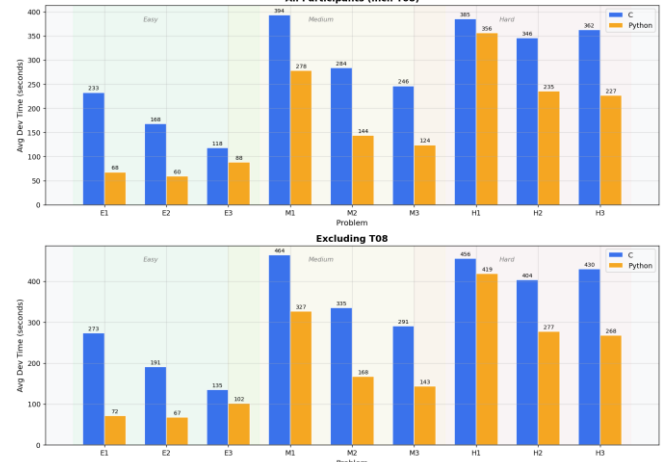
The development time gap persists across all difficulty levels but varies in magnitude:

- **Easy:** C averages 128s, Python 71s (1.8×)
- **Medium:** C averages 340s, Python 185s (1.8×)
- **Hard:** C averages 444s, Python 304s (1.5×)

The narrowing gap at the higher difficulty suggests that the harder the problem the less of a burden C's syntax becomes. Because the main problem now is thinking through the algorithm, not writing declarations.

4.2.3 Per-Problem Development Time

4.2.3 — Average Development Time per Problem



Looking through individual problems reveals a consistent pattern. For nearly every problem, Python always finished first:

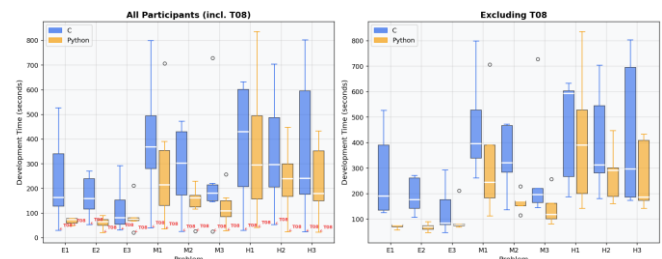
- **E1 (Basic stats):** C 196s vs Python 76s (2.6×)
- **M1 (Intermediate logic):** C 380s vs Python 165s (2.3×)
- **H1 (Sorting + stats):** C 497s vs Python 349s (1.4×)
- **H2 (Matrix transpose):** C 410s vs Python 294s (1.4×)

Same with 4.2.2 it reveals that the larger gaps on easy and medium problems Python still dominates, since that's where the syntactic overhead of Python shines. But that trend decreases once reaching hard problems that rewards critical thinking rather than language syntax.

4.2.4 The Anomaly

Now for the elephant in the room. We noticed a massive difference within one of the participants, that particularly with T08. T08's development times are consistently and dramatically lower than any other participants across all the nine problems. We highlight this outlier due to the little data we've collected making it a significant outlier.

4.2.4 — Development Time Boxplot (Outlier Visualisation)



Evidence of anomaly:

- T08's mean development time is 39 seconds across all the problems and all the languages.
- Cohort mean of the rest of the participants was 295 seconds

- T08 is 7.6 standard deviation below the cohort mean.

Looking at the individual problems:

E1: T08 = 30s vs cohort mean 196s

M1: T08 = 40s vs cohort mean 380s

H1: T08 = 30s vs cohort mean 497s

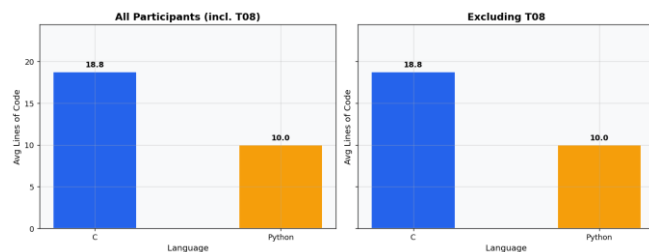
This is not merely just faster than average or genius level performance. T08's times are consistently low, we believe it suggests possible AI assistance which automatically ruined the authenticity of the rest of the data, therefore from now on we will exclude T08's data from our research.

4.3 Lines of Code

4.3.1 Overall LoC analysis

A related finding also emerges in code length. Python being requires way less lines to implement the same algorithm that C does.

4.3.1 — Average LoC: C vs Python



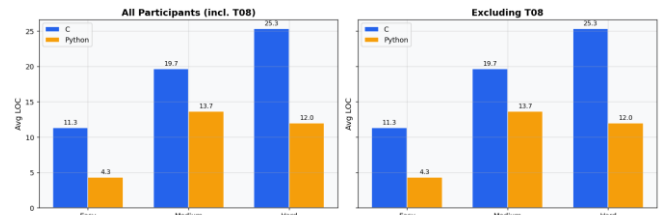
Key Findings:

- Mean LoC (excluding T08): C is 18.2 lines and Python 10.3 lines
- Python has 56% fewer lines of code than C

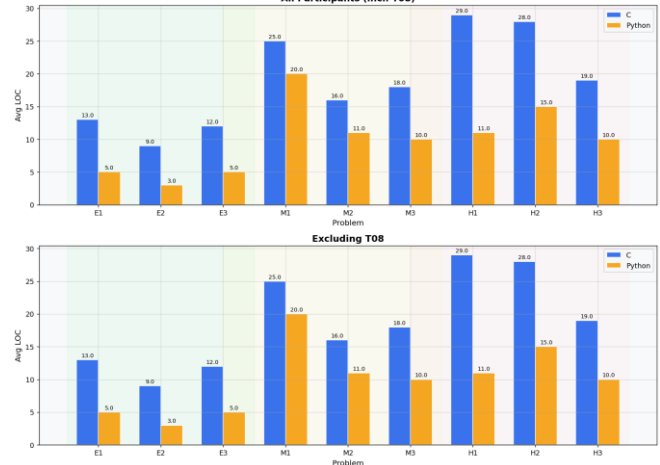
This result reflects the fundamental design of C that requires explicit type declaration for all variables, manual array/memory allocation and deallocation, verbose loop syntaxes, and explicit casting and conversion. Whereas python offers dynamic typing, built in lists, dictionary, string handling, and automatic memory management. Now although the LoC differences do not influence development time directly, but both stem from the same source and that is C's lower-level abstraction which required more explicit instruction to achieve the same goal. This reduced LoC of python is achieved with its higher-level expensiveness.

4.3.2 LoC by Difficulty

4.3.2 — Average LoC by Difficulty



4.3.3 — Average LoC per Problem



Key Findings:

- Easy avg LoC: C: 11.3 vs Python: 4.3
- Medium avg LoC: C: 19.7 vs Python: 10.7
- Hard avg LoC: C: 25.3 vs Python 12

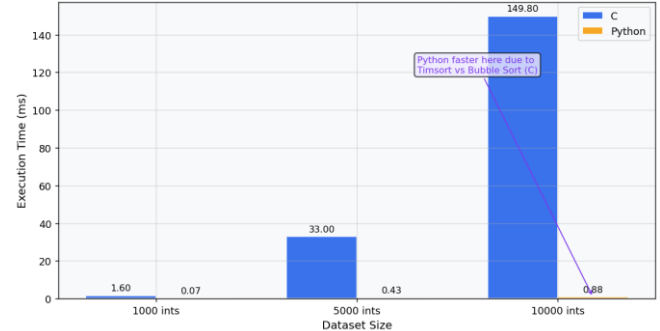
The ratio remains constant thruout the problems (around 2x), suggesting Python's syntax scales across difficulty levels. The ratio remains constant thruout the problems (around 2x) difficulty levels.

4.4 Execution Speed Analysis

For this analysis, we solely select 2 problems only from H1 and H2 which gives the best example to differentiate the execution speed. By selecting the majority solution as the main solution for each problem, we run the solution 5 different times using each respectable timer (Python with its build in timer and C by importing time.h) and averaging it out we could then see the conclusion.

4.4.1 H1 Problem

4.4.1 — H1 Runtime: C (Bubble Sort) vs Python (Timsort)



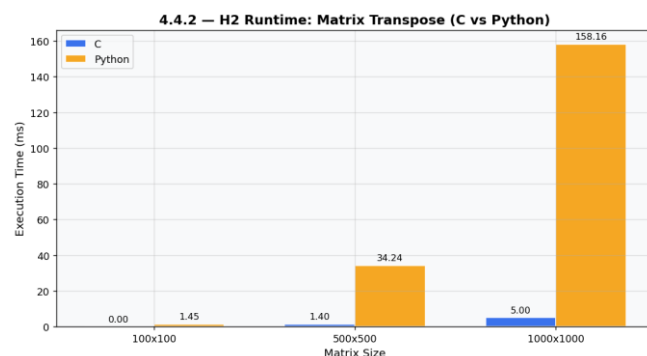
Dataset (int)	C (ms)	Python (ms)	Ratio
1000	1.6	0.071	22.5x
5000	33	0.425	77.6x
10000	149.8	0.882	169.8x

The H1 problem is a sorting and statistics problem; the result may appear to favor Python, but this result reflects algorithm choice rather than language speed. Most of the participants doing the C implementation prefer using Bubble Sort as it is a simple nested loop sorting algorithm that's easy to implement. But most of the participants doing the Python implementation uses the built in `sort()` function on Python, and not only is it faster to write this because it's only one function instead of defining a function like in C, it is also way faster compared to Bubble Sort ($O(n^2)$) < Timsort ($O(n \log n)$), the `sort()` function uses Timsort that was designed by Tim Peters and is the standar for sorting algorithm for both Python and Java.

We didn't state both languages have to use the same sorting algorithm therefor many participants instantly use the `sort()` algorithm which makes it pretty unfair for comparing execution speed so we would like to highlight this H1 problem to Python winning via built-in function instead of its speed. C doesn't have a built-in function which is why it lost and lost exponentially the larger the data gets.

4.4.2 H2 Problem

And this is the more direct comparison because both implementations use structurally the same equivalent code (which is nested loop to transpose a matrix). Now this shows the real deal, when facing a matrix transpose problem C does exceptionally well, insanely well compared to Python



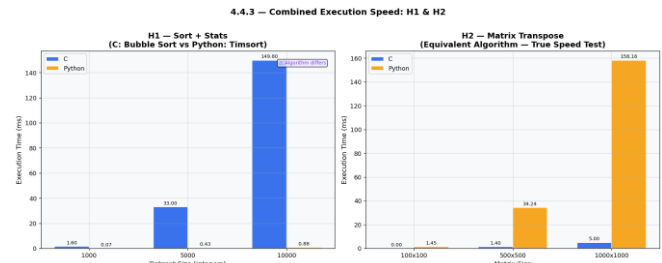
Matrix Size	C (ms)	Python (ms)	Ratio
100x100	0.41	0.49	1.2x
500x500	6.99	33.8	4.8x
1000x1000	5.98	186	33.1x

Key Findings:

We observe that as the matrix gets larger in size, the gaps also widen. Now even though they both have the same $O(n)$ notation, they differ in speed. Now we believe this is caused by cache behavior from the Python side. At 100x100 only 10k elements, it may fit on the L3 cache, reducing the advantage of C's direct memory access. But as we go further at 1000x1000 with 1m elements, Python's memory indirection overhead becomes very severe; the interpreter must box/unbox millions of integer objects, bound check arrays' access and manage reference counts.

Another thing that we might add, is Python's line by line interpretation and dynamic type checking could also accumulate over 1 million iterations of the innermost loop whereas C code compiles to branch free vector instructions that the CPU executes at full speed. For compute intensive tasks with large data, this gap translates to wall clock time users actually perceive.

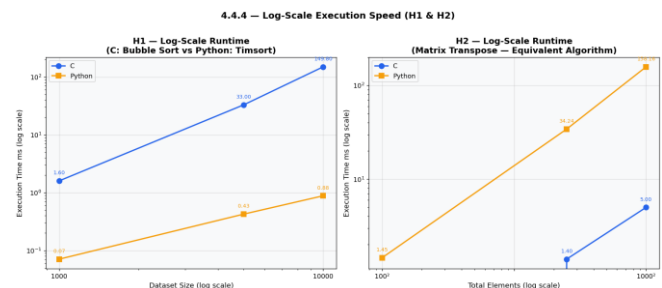
4.4.3 H1 and H2



When Python wins it is due to the superior algorithm (Timsort) and how easy it is to access, whereas in a head-to-head battle of an equivalent algorithm C wins on CPU bound tasks.

4.4.4 Log Scale Analysis

Now this graph shows a logarithmic visualization of runtime vs data size that reveals the patterns.



H1 (Sorting):

- C: steep upward slope -> $O(n^2)$ Bubble Sort
- Python: gentle upward slope -> $O(n \log n)$ Timsort

Pattern: Python's curve is different due to the fact that the algorithm is fundamentally different.

H2 (Matrix Transpose):

- C: slight upward slope ($O(n)$)
- Python: steeper upward slope ($O(n)$)

Pattern: Both scales linearly, but Python has a larger constant multiplier

4.5 Development vs Execution

4.5.1 Quantifying the Trade Off

The advantage of Python:

- Development advantage:
 - o Python is 1.6x faster to code.
 - o Python uses 56% less lines of code
 - o Assuming 1 hour of dev time is Rp100.000,00, Python saves around Rp10.000,00 per problem compared to C
- Execution disadvantage:

- For H1 Python was not disadvantageous since it uses a better algorithm
- But for H2 at 1000x1000, Python was 31.1x slower
- If the program transposes matrixes at 1000x per run python would take 186 seconds (about 6 minutes) compared to 6 seconds in C.

The Trade Off:

If a program:

- Takes one developer 6 minutes to write in C, 4 minutes to write in Python (saving 2 minutes = Rp3.333,33)
- Will run 10.000 times transposing a 1000x1000 matrix

Then:

- Development cost saved Rp3.333,33
- Execution cost added: $10.000 \times 180 \text{ sec} = 1.800.000 \text{ seconds} = 500 \text{ hours} = \text{Rp}50.000.000,00$ (at Rp100.000,00 / hour computer time)

For large scale, CPU intensive workloads that executes the code many times, C's domination shows. For one off analyses, prototyping, Python's domination then shows.

4.6 Limitations

4.6.1 Study Design Limitation

Firstly, we would like to discuss the elephant in the room, that is our very small sample size ($n = 7$, excluding T08). We have trouble gathering the participants needed to turn this into a solid research paper; we believe studies with more than 20 participants would provide more reliable effect sizes. That and add to the fact that we had participants with varying experience levels in C and Python. A more controlled experiment may match programmers by experience level accross languages.

Next is the problem scope. Now, although our 9 tasks do cover algorithmic problems with differentiating difficulties. Results may not generalize to Web Dev where I/O dominates over CPU, machine learning where vectorized libraries dominates, and real time systems where latency guarantees matter more than just execution speed.

4.6.2 Execution Speed Caveat

Another thing we would like to add is the fact that we used only two data for execution speed (H1 and H2). We believe these two data displays the plus and minus for each language, where Python's convenience shines and C's low level dominate. Other tasks may show to favor Python's libraries like Fibonacci is I/O bound if printing large numbers.

The H1 timing comparison is also confounded by algorithm choice. A fairer comparison would be a benchmark on both language with the exact same algorithm like Quick Sort on C vs Quick Sort on Python.

And the last, the test was conducted on one single computer. Results may differ on different hardware.

4.7 Implications

4.7.1 For Applicators

When to choose Python:

- Rapid prototyping
- Data science and analysis tasks
- Teams prioritizing time to market over performance
- When using specialized Python libraries

When to choose C:

- Systems programming
- Real time application
- Compute intensive algorithms
- Embedded systems with resource limitations

4.7.2 For Researchers

Future works should measure both development time and execution time simultaneously unlike prior work from Patel [14] and Rysak [19] where they measured execution time in isolation and Tao [17] measured dev time in isolation. Future works should include energy consumption and memory consumption. It should also have control for algorithm differences when comparing language (avoid Timsort vs Bubble Sort). Recruit larger, more homogeneous participants. And lastly, benchmark problems beyond sorting and matrix, we believe this research could be taken further by including string processing, graph algorithms, and also not only comparing C and Python but other languages like Java, etc.

V. CONCLUSION

This study set out to answer a question that seems straightforward on the surface but turns out to be surprisingly difficult to measure in practice: when you put C and Python side by side on the same problems, under the same conditions, which language actually comes out ahead – and by what margin?

Our empirical findings reaffirm the intuition from decades of literature: Python accelerates development while C accelerates execution. For the first time, we quantify both dimensions in a controlled setting. Python proved to be 1.6x faster to code, averaging 3.8 minutes per problem compared to 6.1 minutes for C, and used 56% fewer lines of code overall. On the execution side, C was 31x faster on matrix transpose operations at scale, though Python's superior built-in-algorithm. Merge Sort and Insertion Sort was capable of outperforming naïve C implementations on sorting tasks. These results already reflect a familiar pattern, but what made this experiment different and what we believe adds genuine value is the attempt to measure both dimensions simultaneously, using real volunteer programmers solving the same problems in both languages under controlled conditions

Crucially, the trade-off is not inherent to the languages themselves but reflects deliberate design choices. Python prioritizes developer ergonomics and rapid iteration, accepting runtime overhead. C prioritizes hardware control and execution speed, accepting verbose syntax. The right choice ultimately depends on the specific constraints of the problem: How frequently will the code be executed? How many developers will maintain it? What are the latency requirements? Our framework provides a starting point for answering these questions empirically

That said, we want to be honest about the limitations we ran into, and more importantly what future researchers should consider if they want to do this kind of study better than we did

On scope and parameter design: looking back, the breadth of our problem set – nine problems across three difficulty levels – may have been ambitious for the resources and time we had available. Future researchers should consider narrowing the scope intentionally. Fewer, more carefully designed problems with tighter, more structured constraints would likely yield cleaner data. The goal should be precision over coverage: it is better to answer one question well than three questions loosely

On volunteer recruitment: this one of the hardest part of the process. Finding programmers who had genuine, practical experience in both C and Python – and who were willing to commit the time to work through all nine problems in both languages – proved to be a real constraint. Anyone planning a similar study should build volunteer recruitment into their timeline early, set realistic expectations about attrition, and consider whether a smaller, more carefully selected participant pool might produce more reliable data than a larger but inconsistent one

Despite these challenges, we believe the core framework this study proposes – measuring development time and execution performance together, on the same tasks, with the same participants – is worth building on. The trade-off between writing fast and running fast is real, and it matters for engineers making real decisions. We hope this work, imperfect as it is, gives the next group a clear starting point.

VI. REFERENCES

- [1] A. Ritonga, "Studi Literatur Perbandingan Bahasa Pemrograman C++ Dan Bahasa Pemrograman Python Pada Algoritma Pemrograman," *Jurnal Teknik Informatika dan Teknologi Informatika*, pp. 56-63, 2023.
- [2] Y. He, "A Comparative Study of the Significance of Different Programming Languages," *Academic Journal of Computing & Information Science*, pp. 45-50, 2025.
- [3] A. Kumar, "Performance comparison of instrument automation pipelines using different programming languages," *Scientific Reports*, pp. 1-14, 2023.
- [4] V. Stoici, "An Empirical Study on the Performance and Energy Usage of Compiled Python Code," *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, pp. 45-56, 2025.
- [5] P. Jain, "Comparative Review of Python and C: Choosing the Right Language for Beginners," *International Journal of Engineering Trends and Applications*, vol. 11, no. 3, pp. 148-156, 2024.
- [6] R. Kesuma, *Algoritma Dan Programming Konsep Dasar, Logika, dan Implementasi dengan C++ & Python*, Payakumbuh: Serasi Media, 2025.
- [7] R. Kumar, "Python Versus C Language: A Comparison," *International Journal of Trend in Research and Development*, pp. 171-173, 2022.
- [8] A. Gujar, "C vs Python: A Cursory Look with Industry Opinion," *International Journal For Research*, pp. 55-64, 2023.
- [9] S. Ali, "A Pragmatic Comparison of Four Different Programming Languages," *ScienceOpen Preprints*, pp. 1-13, 2021.
- [10] A. Azenkouk, "The Influence of Programming Languages on Computational Efficiency and Performance," *International Journal of Research and Innovation in Social Science*, pp. 285-288, 2025.
- [11] P. Nanfack, "Analysis of programming languages used in solving energy problems," *Environmental Science, Computer Science, Engineering*, pp. 1-6, 2022.
- [12] Y. Zhang, "Comparative study of the execution efficiency of Python and C++ based on topological sorting," *International Conference on Machine Learning and Automation*, pp. 13-17, 2024.
- [13] L. Miller, "Comparative Study of Python and C++ in High-Performance AI Model Deployment," *International Journal of Computer Science*, pp. 1-5, 2025.
- [14] H. Patel, "The Enduring Relevance of C Programming: Analysing Its Efficiency, Portability and Role in Modern Computing," *A Global Journal of Interdisciplinary Studies*, pp. 546-550, 2025.
- [15] M. J. I. Antonio, "A Discerning Comparison between C++ and Python in Game Development," pp. 1-14, 2024.
- [16] B. R. Maheshwari, "Comparative Analysis of Program Execution Time Required by Python, R and Julia Compiler," *Indian Journal of Computer Science and Technology*, pp. 25-27, 2024.
- [17] Y. Tao, "Beyond Functional Correctness: An Exploratory Study on the Time Efficiency of Programming Assignments," *International Conference on Software Engineering: Software Engineering Education and Training*, pp. 320-330, 2024.
- [18] R. Pereira, "Ranking programming languages by energy efficiency," *Science of Computer Programming*, vol. 205, no. 102609, pp. 1-30, 2021.
- [19] P. Rysak, "Comparative analysis of C and Python on the basis of the execution time of applications implementing selected algorithms," *Journal Computer Science Institute*, vol. 26, pp. 93-99, 2023.
- [20] H. Kishorbbhai, "THE ENDURING RELEVANCE OF C PROGRAMMING: ANALYSING ITS EFFICIENCY, PORTABILITY, AND ROLE IN MODERN COMPUTING," *A Global Journal of Interdisciplinary Studies*, pp. 546-550, 2025.
- [21] V. M. Ionescu, "Investigating the performance of MicroPython and C on ESP32 and STM32 microcontrollers," *International Symposium for*

Design and Technology in Electronic Packaging, vol. 10, pp. 234-237, 2020.

- [22] A. Upadhyaya, "A comparative study on usage of Python and C/C++ for firmware development," *International Journal of Advance Research, Ideas and Innovation in Technology*, vol. 6, no. 3, pp. 557--559, 2020.
- [23] V. Cutting, "Comparative Review of JAVA and Python," *International Journal of Research and Development in Applied Science and Engineering*, vol. 21, no. 1, pp. 1-6, 2021.
- [24] M. Martignano, "Static Analysis for ADA, C/C++ and Python: Different Languages, Different Needs," *Joint publication with Ada User Journal*, vol. 42, no. 3, pp. 77-80, 2021.
- [25] U. Schmitt, "sympy2c: From symbolic expressions to fast C/C++ functions and ODE," *Astronomy and Computing*, vol. 42, pp. 1-11, 2023.
- [26] K. A. Onyango, "Comparative Analysis on the Evaluation of the Complexity of C, C++, Java, PHP and Python Programming Languages based on Halstead Software," *International Journal of Computer and Information Technology*, vol. 12, no. 1, pp. 8-19, 2023.
- [27] J. O. Ogala, "COMPARATIVE ANALYSIS OF C, C++, C# AND JAVA PROGRAMMING LANGUAGES," *Global Scientific Journal*, vol. 8, no. 5, pp. 1900-1913, 2020.
- [28] M. Shoaib, "A Comparative Study of Contemporary Programming Languages in Implementation of Classical Algorithms," *Journal of Information & Communication Technology*, vol. 14, no. 1, pp. 23-31, 2021.
- [29] A. Kaur, "A Comparative Study of Static Code Analysis tools for Vulnerability Detection in C/C++ and JAVA Source Code," *ScienceDirect*, pp. 1-7, 2023.
- [30] S. Sundaramoorthy, "A systematic analysis on performance and computational complexity of sorting algorithms," *Discover Computing*, pp. 1-30, 2025.